FINAL-YEAR B.TECH PROJECT · COMPLETE DOCUMENTATION

Intelligent Transformer Health Monitoring Using Dissolved Gas Analysis (DGA) and Machine Learning

From classical diagnostic standards to a calibrated, explainable machine-learning classifier, a retrieval-augmented LLM diagnostic agent, and a live conversational monitoring dashboard.

PREPARED BY	Ankit Raj
PROGRAMME	B.Tech Electrical Engineering (Final Year), KIIT University, Bhubaneswar

DATASET	IEEE DataPort — DOI: 10.21227/27vy-h479
STANDARDS	IEC 60599:2022 · IEEE C57.104-2019 · CIGRE TB 761
REPOSITORY	github.com/AR0714/transformer-health-dga
LIVE DASHBOARD	transformer-dga.netlify.app

Document version 1.0 · Compiled from Project Handbooks Parts 1–4

Table of Contents

1	Executive Summary	3
2	Problem Statement and Motivation	4
3	Overall System Architecture	5
4	Part 1 — Building the Foundation (Classical Methods + ML Baseline)	7
5	Part 2 — From Working Model to Industrial-Grade Product	16
6	Part 3 — LLM-Powered Diagnostic Agent (RAG over IEC 60599)	21
7	Part 4 — Conversational Digital Twin + Live Web Dashboard	26
8	GridPulse — The Production Front-End	33
9	Consolidated Results	34
10	Honest Limitations	35
11	Final Repository Structure	36
12	How to Run Everything	39
13	Standards and References	40
14	Credits	41

1 Executive Summary

A power transformer costs USD 2–10 million and takes 18–24 months to replace. When an internal fault develops — overheating, partial discharge, arcing — the insulating mineral oil breaks down and releases specific dissolved gases into solution. Reading those gases is the single most information-dense, lowest-cost way to catch a transformer fault before it becomes a catastrophic failure, and it is a technique with a sixty-year history in power engineering (Dissolved Gas Analysis, or DGA).

This project takes that diagnostic technique from a set of classical hand-calculations through four complete engineering phases, each one documented in its own handbook, and ending in a live, cinematic, AI-assisted monitoring dashboard that a recruiter, professor, or utility engineer can open in a browser today:

- **Part 1** builds the foundation: classical diagnostic methods coded from scratch (Key Gas, IEC 60599 ratios, Duval Triangle), a calibrated XGBoost classifier that beats them, and the first working product (risk ranking + Streamlit dashboard).
- **Part 2** turns that working model into an industrial-grade product: a six-method classical consensus engine, an abstention layer that refuses to guess when uncertain, a full data-leakage audit, rigorous statistical evaluation with confidence intervals, automated per-transformer PDF/HTML reports, a data-quality validation layer, and a standalone open-source Python package (`pydga`) published to PyPI.
- **Part 3** adds a Retrieval-Augmented Generation (RAG) layer: a local large language model (Ollama, Llama 3.2) grounded in the actual text of IEC 60599:2022 and IEEE C57.104-2019, so an engineer can ask follow-up questions in plain English and get answers that cite the standard, not hallucinations.
- **Part 4** turns the system into a conversational digital twin: a chatbot that understands intent (health / voltage / gas / load / action-advice), remembers conversation context, adapts its vocabulary to a layperson or an engineer, and is wrapped in a live web dashboard with an animated transformer graphic, real-time gauges, and an embedded chat panel — deployable to the public internet.

The result that is publicly live today — **GridPulse** — is the polished, cinematic React + Three.js realization of that Part 4 vision: a portfolio-grade, full-screen, glassmorphism dashboard with a particle-based 3D transformer winding, a live Health Index orb, dissolved-gas breakdowns, SHAP-based model reasoning, a UNIT-07 substation profile, and a Groq-powered conversational assistant — all built on top of the research documented below.

Headline result: a calibrated XGBoost classifier reaches **80.0% accuracy** on a sealed, stratified 70-row held-out test set, beating the classical Duval Triangle baseline of **57.1%** by **+23 percentage points**, with SHAP confirming the model has learned the real underlying chemistry (acetylene, C_2H_2, is the dominant driver of arcing detection) and a fully leakage-audited, honestly reported evaluation methodology.

---

2 Problem Statement and Motivation

A power transformer is one of the most expensive single assets in an electrical grid, and also one of the hardest to replace quickly — lead times of 18 to 24 months are typical for a large unit, with costs in the USD 2–10 million range. When an internal fault begins — partial discharge in the insulation, a loose connection overheating, or a flashover arcing through the windings — the transformer does not fail instantly. Instead, the fault slowly breaks down the oil and the cellulose paper insulation, releasing specific gases (hydrogen, methane, ethane, ethylene, acetylene, carbon monoxide, carbon dioxide) into the oil in characteristic ratios that depend on the *type* and *severity* of the fault.

This has been known since the 1970s, and a family of classical interpretation methods exists — Key Gas, Rogers Ratios, Doernenburg Ratios, the IEC 60599 ratio method, and the Duval Triangle/Pentagon — all of which are hand-calculable from five to seven gas readings. These methods are codified in international standards (IEC 60599, IEEE C57.104, CIGRE Technical Brochure 761) and are still the backbone of condition-based maintenance programs worldwide.

The opportunity this project pursues is not to replace these classical methods, but to:

1. **Implement them faithfully** as real, tested, standards-literate code (not black-box ML from the start) — because understanding the physics first is what separates an engineer who can defend every decision from someone who fit a model to data they don't understand.
2. **Beat them** with a properly validated, calibrated, explainable machine-learning classifier trained on physics-informed features.
3. **Operationalize** that classifier into something a utility engineer could actually act on: a risk-ranked fleet view, automated per-transformer reports, an abstention mechanism for uncertain cases, and finally a conversational interface that answers "is my transformer okay?" in plain English, grounded in the actual standard text.

This is deliberately framed as a diagnosis-and-triage layer on top of classical standards, not a replacement sensor platform or a claim of predicting failures — a distinction the documentation below is careful to preserve throughout, because overclaiming is the most common way an otherwise good ML project loses credibility with people who actually work in power systems.

---

3 Overall System Architecture



4 Part 1 — Building the Foundation (Classical Methods + ML Baseline)

Subtitle from the original handbook: "Intelligent transformer health monitoring from dissolved gas analysis — every step, what to do, why it matters, and how to know you have finished it."

Part 1 is organized as **25 steps across six parts (Part 0 through Part 5)**, written so that someone with zero prior Python or ML experience could execute it, in build order rather than textbook order. Each step in the source handbook carries four fixed sub-sections — **Why This Step Exists**, **What To Do**, **Done When**, and **The Mistake Almost Everyone Makes Here** — which are preserved below because they encode not just what was built but why each choice was made, which is exactly the material that later became the project's `DECISIONS.md` engineering log.

Part 0 — Before You Write a Single Line of Code (Steps 1–5)

Step 1 — Install Python (the whole toolbox at once). Python alone is just a language; the project also needs pandas for tables, scikit-learn for models, and Jupyter to work interactively. Installing these one-by-one is where many beginner setups go wrong. The handbook's instruction: install the Anaconda distribution (anaconda.com), accept all installer defaults, then launch Anaconda Navigator → JupyterLab. *Mistake to avoid:* installing plain Python from python.org and fighting package installation for two days — not a part of the project worth being clever about. **Done when:** a JupyterLab tab is open and `2 + 2` typed into a cell and run with Shift+Enter shows `4`.

Step 2 — Build the folder before you fill it. Where files live determines whether a project stays workable by week eight. The handbook specifies an exact folder skeleton, built by hand before any code is written:

```
transformer-health-dga/
  data/           <- the dataset files go here
  notebooks/      <- exploration, messy thinking
  src/            <- real code that gets reused
    classical/    <- Key Gas, ratios, Duval
  models/         <- saved trained models
  reports/        <- figures and results
```

Mistake to avoid: putting code in notebooks forever — notebooks are for exploring; anything run more than twice moves into `src/` as a proper file. **Done when:** the six folders exist and are empty.

Step 3 — Start Git on day one, not day forty. Git records every version of the work, so a mistake is never fatal, but the bigger reason given in the handbook is that the GitHub page will be read by people deciding whether to interview the author, and a repository whose commit history shows steady work over weeks says something a single last-day upload cannot. Instructions: create a free GitHub account, install GitHub Desktop, add the local folder as a repository, publish it, and **uncheck the "private" box** so it is public. *Mistake to avoid:* making it

private and forgetting to switch it later; also, never commit large data files — add a `.gitignore` containing the single line `data/`. **Done when:** the empty folder structure is visible at `github.com/yourname/transformer-health-dga`.

Step 4 — Write the README before you write the code. This is called the single highest-leverage habit in the whole handbook ("README-driven development"). Writing down what is being built and why, *before* building it, forces real understanding and means documentation grows beside the project instead of being written in a panic at the end. The initial README is deliberately minimal: a one-line description, a paragraph on why transformer failures are expensive (using the lead-time and replacement-cost figures), a checklist of planned work, and a status line. *Mistake to avoid:* writing "a machine learning project for transformer fault detection" and stopping — that describes the technique, not the problem; lead with the asset and the cost. **Done when:** `README.md` exists and renders on the GitHub page.

Step 5 — Open the decision log, and never close it. Every real choice made in the project — which dataset, which split, which model, what was rejected and why — is worth three lines at the moment it's made, and this single file (`DECISIONS.md`) later becomes the most valuable section of the project's documentation and the source of the strongest interview answers, because it cannot be reconstructed or faked afterward. Format: a date, what was decided, and what was rejected and why, appended forever, never deleted. Example entries from the actual project:

```
## 2026-08-14
Decided: IEEE DataPort DGA package as primary dataset.
Rejected: Kaggle DGA sets - labels unverified, and many
'DGA' sets there are Domain Generation Algorithm
(cybersecurity), a completely different problem.

## 2026-08-16
Decided: stratified k-fold, not a plain split.
Why: T2 class has ~21 samples; a random split could
put nearly all of them in one fold.
```

Mistake to avoid: deciding to "remember it and write it up later" — the details that make this file valuable are gone within two days. **Done when:** `DECISIONS.md` exists with at least one entry.

Part 1 — Meet the Data (Steps 6–8)

Step 6 — Get the dataset, and avoid the name trap. The project uses the **IEEE DataPort DGA dataset**, which packages together a training set, an unseen real-world test set, and the IEC TC10 benchmark that published papers use — meaning results are directly comparable to the literature. A genuine hazard flagged here: the letters "DGA" mean something completely different in cybersecurity (Domain Generation Algorithm), and public dataset search engines are full of the wrong kind. **Done when:** CSV/spreadsheet files sit in `data/` and opening one shows gas columns with a fault label.

Step 7 — Load it and look at it. Before any analysis, the actual shape of the data must be seen with real pandas work:

```
import pandas as pd
df = pd.read_csv('../data/your_file.csv')
print(df.shape)
print(df.columns.tolist())
df.head(10)
df.isnull().sum()
```

Mistake to avoid: skipping straight to modelling — everything not noticed here becomes a bug that can't be explained later. **Done when:** row/column counts and physical column meanings can be stated out loud.

Step 8 — Count the classes, and take the count seriously. This single number shapes every later decision. The project's representative dataset is **376 samples**, with **113** in the majority high-energy-discharge class but only **21** in the rarest class, T2 (thermal 300–700 °C). That imbalance is why plain accuracy is misleading, why the split must be stratified, and why deep learning would be the wrong tool. The exercise: compute what accuracy a model would get by always predicting the most common class (the "lazy-model floor" that any real model must clear), and record that number in `DECISIONS.md`.

```
df['fault_type'].value_counts()
df['fault_type'].value_counts(normalize=True)
```

Mistake to avoid: seeing an imbalanced dataset and immediately reaching for SMOTE to synthetically balance it — synthetic gas mixtures can be physically impossible; understand the imbalance first, treat it later, carefully, if at all.

Part 2 — Put the Physics into Code (Steps 9–13)

"The part that makes your project different from everyone else's. Most people skip straight to machine learning. Doing this first gives you a baseline to beat, features for the model, and the domain credibility that actually gets you interviewed."

Step 9 — Implement the Key Gas method. The simplest classical method: which gas is dominant? Dominant hydrogen suggests partial discharge; dominant ethylene suggests a hot thermal fault; dominant acetylene means arcing; high carbon monoxide points at paper degradation.

```
def key_gas(h2, ch4, c2h6, c2h4, c2h2, co):
    gases = {'PD': h2, 'T1/T2': ch4, 'T2': c2h6,
             'T3': c2h4, 'D2 arcing': c2h2, 'paper': co}
    return max(gases, key=gases.get)
```

Mistake to avoid: treating this as the finished diagnostic — Key Gas is the fastest field check and the least precise method there is, a starting point, not an answer. **Done when:** one row of real gas values returns a sensible fault name.

Step 10 — Implement the IEC 60599 ratio method. Ratios are more informative than single values because they compare energies rather than raw amounts. IEC 60599 uses three: acetylene/ethylene, methane/hydrogen, ethylene/ethane — the international standard, and the first piece of genuine standards literacy in the project.

```
def iec_ratios(h2, ch4, c2h6, c2h4, c2h2):
    def safe(a, b):
        return a / b if b > 0 else None
    return {
        'c2h2_c2h4': safe(c2h2, c2h4),
        'ch4_h2':    safe(ch4, h2),
        'c2h4_c2h6': safe(c2h4, c2h6),
    }
    # then map the three ratios to PD / D1 / D2 / T1 / T2 / T3
    # using the boundary table from IEC 60599
```

Two things handled carefully: division by zero when a gas reads zero, and the fact that this method is explicitly allowed to return "no decision" at all. *Mistake to avoid:* applying ratios when all the gases are tiny — at very low concentrations a small measurement error swings a ratio wildly, producing noise rather than signal.

Step 11 — Implement the Duval Triangle. *"The centrepiece of the classical work, and the most satisfying thing to build. It takes three gases, turns them into a single point on a triangle, and reads off the fault zone. It is pure arithmetic and geometry — no machine learning at all — which makes it ideal Python practice that happens to be real industrial code."*

```
def duval_coords(ch4, c2h4, c2h2):
    s = ch4 + c2h4 + c2h2
    if s == 0:
        return None
    a, b, c = ch4/s, c2h4/s, c2h2/s   # fractions
    x = 100*b + 50*a
    y = 86.6*a
    return x, y

from matplotlib.path import Path
zone = Path(ZONE_VERTICES['D2'])
inside = zone.contains_point((x, y))
```

Convert the three gases to percentages of their own total, map to Cartesian X/Y coordinates (X = 100b + 50a, Y = 86.6a, where a/b/c are methane/ethylene/acetylene fractions), then use a point-in-polygon test against the standard's zone boundaries. *Mistake to avoid:* forgetting that the triangle can never say "healthy" — every point lands in some zone, so the standards must be checked first to establish that something is abnormal before using the triangle to identify what.

Step 12 — Check the code against published examples. At this point there are three methods and no proof any of them are correct. The exercise: find worked examples (from the handbook or Duval's own published papers) where both the gas values and the correct diagnosis are given, and run the code against them — every case must

match exactly, or a bug has been found. **Done when:** at least five published examples produce exactly the diagnosis the source states. *Mistake to avoid:* assuming code that runs without error is code that is correct — these two things have very little to do with each other.

Step 13 — Score the classical baseline. This is the number the machine learning must beat, and it is what turns the project from a demo into an argument — without it, a model scoring 88% means nothing, because there is no way to tell whether that is impressive or worse than arithmetic that has existed since the 1970s. Each classical method is run across the whole dataset, scored for overall accuracy, macro-F1, and — importantly — how often the ratio methods return "no decision" at all, all written into the decision log.

```
from sklearn.metrics import accuracy_score, f1_score
print(accuracy_score(y_true, y_duval))
print(f1_score(y_true, y_duval, average='macro'))
no_decision = sum(1 for p in y_iec if p is None)
print('no decision rate:', no_decision / len(y_iec))
```

Part 3 — Machine Learning (Steps 14–18)

"Five steps. Note that it arrives here, more than halfway in, which is exactly the point that impresses people who build these systems for a living."

Step 14 — Engineer the features. The model is never handed raw gas numbers alone — the IEC ratios and Duval coordinates built in Part 2 encode decades of physics, and handing those to a model as extra columns nearly always beats raw values alone. This is where domain knowledge becomes measurable performance, and it is the real advantage over a data scientist with no electrical background. The dataset grows from five columns to roughly thirteen: three IEC ratios, Duval X and Y, and three gas percentage fractions.

```
df['c2h2_c2h4'] = df['C2H2'] / df['C2H4'].replace(0, pd.NA)
df['ch4_h2']    = df['CH4'] / df['H2'].replace(0, pd.NA)
df['c2h4_c2h6'] = df['C2H4'] / df['C2H6'].replace(0, pd.NA)
coords = df.apply(lambda r: duval_coords(
    r['CH4'], r['C2H4'], r['C2H2']), axis=1)
df['duval_x'] = [c[0] if c else None for c in coords]
df['duval_y'] = [c[1] if c else None for c in coords]
```

Mistake to avoid: adding features without handling the division-by-zero cases first — silent infinities will poison the model.

Step 15 — Split the data the way the field will judge you. This decides whether the accuracy number is real or an illusion, and getting it wrong is the single most common way portfolio projects quietly overstate themselves. With a rare class of around twenty samples, a careless split can put nearly all of them on one side, making the reported score meaningless.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

A **stratified** split is used, which keeps each class in the same proportion on both sides, and it is done **before** any scaling or transformation — scaling first leaks test-set information into training and inflates the score by a bug rather than skill. *Mistake to avoid:* exactly that — scaling before splitting looks harmless and invalidates the entire result.

Step 16 — Train one boring model first. Resist starting with the most powerful algorithm; a simple model fully understood is worth more right now than a strong one that can't be explained, because the job in an interview is to defend every choice. Random forest is the right first pick — it handles small tabular data well, needs almost no tuning, and reports which features mattered.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
model = RandomForestClassifier(
    n_estimators=300, class_weight='balanced', random_state=42)
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

Mistake to avoid: reading only the accuracy line — look at per-class recall, particularly for the rarest class, which is where a model quietly fails.

Step 17 — Cross-validate, because one split is not evidence. With only a few hundred rows, a single train/test split is largely luck — a different random seed can move the score several points. Stratified five-fold cross-validation runs the experiment five times over different slices and reports the mean and spread, scoring on macro-F1 (so rare classes count as much as common ones) rather than accuracy.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=cv, scoring='f1_macro')
print(f'{scores.mean():.3f} +/- {scores.std():.3f}')
```

Mistake to avoid: tuning settings, checking the test set, tuning again — that is training on the test set through the back door, and the resulting score is fiction.

Step 18 — Try XGBoost, then choose honestly. Gradient-boosted trees usually match or slightly beat random forest on small tabular problems, so it's worth trying — but the real lesson of this step is the decision at the end: if the more complex model gains half a point of F1 and loses interpretability, choosing the simpler one and explaining why is a stronger answer than chasing the bigger number.

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=300, max_depth=4,
                    learning_rate=0.1, random_state=42)
scores = cross_val_score(xgb, X, y, cv=cv, scoring='f1_macro')
```

Neural networks are explicitly resisted — with a few hundred rows they overfit and calibrate badly, and being able to say that clearly is itself worth marks in an interview. *Mistake to avoid:* assuming the more complicated model must be better — on four hundred rows it very often is not.

Part 4 — Make It Trustworthy (Steps 19–21)

"Three steps that turn a classifier into something a reliability engineer could actually act on. This part is what separates your project from a notebook."

Step 19 — Calibrate the probabilities. A model's raw confidence numbers are usually wrong out of the box — it can say 90% when it is right only 60% of the time — and this matters enormously here because the whole point of the system is ranking transformers by risk, and ranking by dishonest probabilities produces a dishonest priority list.

```
from sklearn.calibration import CalibratedClassifierCV
calibrated = CalibratedClassifierCV(model, method='isotonic', cv=5)
calibrated.fit(X_train, y_train)
probs = calibrated.predict_proba(X_test)
```

A calibration curve (predicted probability on one axis, observed frequency on the other) is plotted and saved; a well-calibrated model sits near the diagonal. *Mistake to avoid:* building the risk ranking before calibrating — the ranking is only as honest as the probabilities beneath it.

Step 20 — Add SHAP explanations. No engineer will act on a black box, and no auditor will accept one. SHAP shows, for any single prediction, which gases pushed the answer which way — turning "the model says D2" into "acetylene at 33 ppm drove this toward high-energy arcing," which is both what makes the system usable and excellent interview material.

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0], data=X_test.iloc[0]))
```

Mistake to avoid: generating the plots without reading them — if SHAP says a feature considered physically irrelevant is dominating, that is a finding worth investigating, not a picture to file away.

Step 21 — Read the confusion matrix like a diagnostician. Overall scores hide the interesting part. The confusion matrix shows exactly which faults get mistaken for which — and when the confusions are physically sensible, that is evidence the model learned something real.

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(
    model, X_test, y_test, xticks_rotation=45)
plt.savefig('../reports/confusion_matrix.png', dpi=150)
```

The diagnostic question: are mix-ups between adjacent fault types (like T2 confused with T3, where the underlying physics genuinely overlaps) expected, or is something absurd happening (partial discharge called high-energy arcing), which would mean something is wrong upstream? *Mistake to avoid:* reporting the matrix as a picture without interpreting it — the interpretation is the whole value.

Part 5 — Make It a Product (Steps 22–25)

"Four steps. A notebook is a record of your thinking; a product is something another person can use. The gap between them is where most portfolios stop and where yours should not."

Step 22 — Build the fleet risk ranking. This is the step that changes what the project *is*. Classifying one sample is a technical exercise; ranking five hundred transformers so a manager knows which to inspect on Monday is a business tool — and it is precisely what commercial products in this space actually ship.

```
df['p_serious'] = probs[:, serious_class_index]
df['risk'] = df['p_serious'] * df['severity_weight']
fleet = df.sort_values('risk', ascending=False)
fleet[['unit_id', 'predicted_fault', 'p_serious', 'risk']].head(20)
```

The scoring formula is deliberately kept simple and defensible — every term must be justifiable. *Mistake to avoid:* inventing an elaborate weighted formula that cannot be justified; simple and defensible beats sophisticated and arbitrary, every time.

Step 23 — Wrap it in a dashboard. A recruiter will not run a notebook. A working interface they can click through — or even a short screen recording of one — communicates in fifteen seconds what a notebook cannot communicate at all. Streamlit turns Python into a web app with no web-development skill required.

```
pip install streamlit
streamlit run src/app/streamlit_app.py
```

Three things built: a box to enter gas values that returns a diagnosis with probability, a Duval triangle plot with the sample marked on it, and the fleet risk table. *Mistake to avoid:* spending a week making it beautiful — three working features beat ten half-finished ones, and nobody is grading the colour scheme.

Step 24 — Rewrite the README as the real thing. Now there are finally results, limitations, and decisions worth writing down. This is the document that determines whether anyone reads the code at all, and the section most portfolios omit — an honest limitations section — is the one that earns the most respect from experienced engineers. The actual limitations written into the project:

```
## Limitations
- Dataset is ~400 labelled samples drawn from overlapping
  public sources. Reported figures are dataset-specific and
  should not be read as field performance.
- Rarest class (T2) has 21 samples; per-class recall there
  is correspondingly unstable.
- No time-series data, so trend features - which the
  standards weight heavily - are absent.
- Not validated against any physically inspected transformer
  outside the benchmark set.
```

Mistake to avoid: hiding the limitations to make the project look stronger — experienced engineers read that section first, and its absence is louder than anything in it.

Step 25 — Publish, and write about it properly. The work only counts if it is visible — but *how* it's presented matters as much as the work itself: a post claiming 94% accuracy with no context reads as naive, while a post explaining the margin beaten over a stated classical baseline on a stated dataset with stated limits reads as an engineer. Everything is pushed to GitHub, the repository link added to a LinkedIn Featured section (not buried in a post body, where external links get suppressed), and the write-up leads with the problem and the classical-baseline comparison — not the headline accuracy number. *Mistake to avoid:* leading with the accuracy number — lead with the problem and the comparison against the classical baseline; that framing is what a hiring manager in this field is actually looking for. **Done when:** the repository is public, complete, and linked from the profile.

> **A note on how long this takes** (from the original handbook): Part 0 is one evening. Part 1 is a few days, mostly spent learning pandas. Part 2 is the longest stretch and the most valuable — expect a week or two, learning Python and implementing standards simultaneously. Parts 3 and 4 move faster since scikit-learn does most of the work. Part 5 is a few days. If a step takes three times longer than expected, that is normal, not a sign of doing it wrong.

---

5 Part 2 — From Working Model to Industrial-Grade Product

Subtitle: "A step-by-step engineering handbook for building an industrial-grade DGA condition-monitoring system — calibrated, explainable, consensus-driven, and honestly evaluated."

This phase is explicitly grounded in deep research into **commercial DGA platforms** (GE Vernova Kelman, Hitachi TXpert, Camlin TOTUS, Reinhausen TESSA) and academic literature (IEC 60599, IEEE C57.104, CIGRE TB 761, plus Kapoor & Narayanan's 2023 data-leakage taxonomy and Sutikno et al.'s 2024 consensus-method research), and is organized as **10 steps**.

An explicit **"do not add"** list was kept throughout Part 2, listing things the data genuinely cannot support and that would require inventing data: time-series trending, remaining-useful-life (RUL) prediction, a digital twin, real-time streaming, dollar-value risk figures, deep learning (CNN/LSTM), full multi-parameter Health Index, CO/CO_2-based paper-fault diagnosis, or financial-risk modelling. This discipline — refusing to claim more than the evidence supports — runs through the whole of Part 2.

Step 1 — Finish and push Part 1 (the gate). Before any Part 2 work begins, the Part 1 repository must actually look finished: all artifacts (the ML notebook, classical method scripts, calibration/SHAP/confusion-matrix figures, fleet risk CSV, dashboard HTML, `DECISIONS.md`, `requirements.txt`) committed in one clean commit, and the README rewritten to the "real thing" standard from Part 1 Step 24 — header with badges, the problem stated with real figures, the required results table (Classical Duval 57.1%, Random Forest 78.6%, XGBoost calibrated 80.0%, ECE 0.15→0.10), an architecture diagram, a dashboard screenshot, 4–6 key findings, and an explicit evaluation-honesty section. **Verify:** visiting the GitHub page in a private browser window shows the results table, a dashboard screenshot, and a non-empty commit history — "the gate is open."

Step 2 — Multi-method classical consensus engine (the signature differentiator). Directly inspired by Camlin's commercial "DGA Matrix" feature, which markets itself as simplifying interpretation "by bringing all leading diagnostic methods together in one clear, colour-coded view," and grounded in Sutikno et al. (2024), which found that five classical methods can disagree on the very same sample. The engine implements and runs **six methods simultaneously** on one gas sample: - **(a) Rogers Ratio Method** — three IEC 60599 ratios (CH_4/H_2, C_2H_2/C_2H_4, C_2H_4/C_2H_6), each coded into a 0/1/2 range, combined into a lookup table mapping to a fault. - **(b) Doernenburg Ratio Method** — four ratios, applicable only when at least one gas exceeds its IEC 60599 L1 threshold; otherwise returns "Below L1 — no diagnosis." - **(c) Duval Pentagon 1** — extends the Triangle to use all five gases as percentages mapped into a five-sided coordinate space, covering more fault zones (including "S," stray gassing). - **(d) A consensus function** that collects every method's verdict, computes an agreement percentage via majority vote, and compares the consensus against the calibrated ML prediction — returning a structured result consumed by the dashboard and reports.

```
def run_consensus(H2, CH4, C2H6, C2H4, C2H2, ml_proba=None, ml_pred=None):
    results = {
        'Key Gas': predict_key_gas(H2,CH4,C2H6,C2H4,C2H2),
        'IEC 60599 Ratios': predict_iec(H2,CH4,C2H6,C2H4,C2H2),
        'Duval Triangle 1': predict_duval(H2,CH4,C2H6,C2H4,C2H2),
        'Rogers Ratios':    rogers_ratio(H2,CH4,C2H6,C2H4,C2H2)[0],
        'Doernenburg':      doernenburg(H2,CH4,C2H6,C2H4,C2H2),
        'Duval Pentagon 1': duval_pentagon(H2,CH4,C2H6,C2H4,C2H2),
    }
    # majority vote → consensus, agreement_pct, and comparison vs ML
```

Verify: consensus runs on all 70 test rows without error; on clear D2 cases most methods agree; on boundary PD/D1 cases the *disagreement* is surfaced rather than hidden — which is exactly the information a real engineer needs.

Step 3 — Abstention / "engineer-review" layer (the trust layer). Directly inspired by Reinhausen TESSA's own stated philosophy: *"We are giving a diagnostic… the asset manager has to make his own decisions."* IEC 60599 ratios already have a built-in "no decision" state when a mixture falls outside the coded boundaries; this step brings the ML model into line with that same humility. When the calibrated model's maximum class probability falls below a threshold (default **0.50**), the system outputs *"Insufficient confidence — engineer review required"* instead of forcing a guess.

```
def predict_with_abstention(proba_row, threshold=ABSTAIN_THRESHOLD):
    if proba_row.max() < threshold:
        return -1, 'Insufficient confidence – engineer review required'
    return proba_row.argmax(), FAULT_NAMES[proba_row.argmax()]
```

A **risk-coverage curve** is then plotted — as the threshold is lowered (answering more cases), how does accuracy on the answered cases change — visualizing the trade-off explicitly and grounding the choice in the selective-prediction literature (Chow's rule; El-Yaniv & Wiener's risk-coverage framework). **Verify:** the curve slopes upward from right to left (confirming abstention helps); at full coverage, accuracy matches the Part 1 baseline (80%); at ~85% coverage accuracy rises to ~85–88%.

Step 4 — Leakage-aware evaluation audit (publishable honesty). Grounded directly in Kapoor & Narayanan (2023, *Patterns* 4(9):100804, Princeton), which surveyed 22 review papers and found data leakage "across 17 scientific fields, collectively affecting 294 papers and in some cases leading to wildly overoptimistic conclusions." This project's own audit discovered that **41 of 49 rows (84%)** in the IEC TC10 benchmark set were exact duplicates of rows already in the training data — and documenting and correcting this is treated as a feature, not a weakness, since most comparable projects simply never check.

```
train_tuples = set(map(tuple, df_clean[gas_cols].round(4).values))
iec_tuples = [tuple(r) for r in df_iec[gas_cols].round(4).values]
leaked_mask = [t in train_tuples for t in iec_tuples]
n_leaked = sum(leaked_mask)
print(f'IEC TC10 rows overlapping with training: {n_leaked}/{len(df_iec)}')
# → 41/49 (84%)
df_iec_clean = df_iec[[not m for m in leaked_mask]].copy()
# → 8 genuinely unseen rows remain
```

After deduplicating the two exact-duplicate training rows and retraining, the model is re-evaluated on three separate slices: the original sealed 70-row test (unaffected, confirming it was already clean), the full 49-row IEC TC10 set *with* leakage (inflated), and the **8 genuinely unseen** IEC TC10 rows (honest, though n=8 is explicitly flagged as too small for strong claims). An "Evaluation Honesty" markdown cell is written directly into the notebook stating all of this plainly — described as "a level of evaluation transparency rare in student projects, and what separates a credible engineering result from an ML demo."

Step 5 — Deeper model comparison with honest metrics (evaluation rigour). Random Forest and XGBoost differed by only 1.4 percentage points on the 70-row test — literally one sample's worth of difference — meaning accuracy alone is noise at this scale. SVM (RBF kernel) and LightGBM are added to the comparison; the primary metric switches to **macro-F1** (treating all seven fault classes equally); and **1000-iteration bootstrap confidence intervals** are computed for every model so that overlapping intervals can be used to prove that small accuracy differences are statistical noise rather than real model superiority.

```
models = {
    'SVM (RBF)': Pipeline([('scaler', StandardScaler()),
                           ('svm', SVC(kernel='rbf', C=10, probability=True))]),
    'Random Forest': RandomForestClassifier(n_estimators=300, random_state=42),
    'XGBoost': XGBClassifier(n_estimators=300, random_state=42),
    'LightGBM': LGBMClassifier(n_estimators=300, random_state=42),
}
# bootstrap_metrics() computes 1000-iteration 95% CIs on accuracy and macro-F1
```

A **per-class recall heatmap** across all four models is also produced — the single most informative view for a seven-class diagnosis problem, showing exactly which faults each model misses (T1/T3 consistently strong across all models; "Normal" the weakest class).

Step 6 — Engineer-facing explainability: SHAP → plain language (communication). Directly inspired by Reinhausen TESSA's product framing of "evidence + probability + recommendation." Raw SHAP plots are unreadable to a field engineer who has never heard of SHAP; this step builds a function that translates SHAP output into human diagnostic prose: a diagnosis line, a confidence percentage (with an abstention flag if below threshold), the top contributing signals stated in plain gas names with direction ("acetylene (C_2H_2) — arcing marker: pushed TOWARD D2"), each gas level compared against its IEC 60599 typical/high thresholds, agreement or disagreement from every classical method, and a recommended action pulled from a fault-meaning

dictionary (e.g., *"D2: High-energy arcing — most critical fault. Potential catastrophic failure. Immediate inspection required. Priority: CRITICAL."*). This is explicitly framed as "what real systems do, and what an interviewer at GE Vernova or Hitachi Energy will ask about."

Step 7 — Automated per-transformer report generation (product feature). Directly inspired by Reinhausen TESSA's "automated reports" and GE Vernova Perception's per-asset health reports. For each of the 70 test transformers, a self-contained HTML report is generated containing: a gas-profile bar chart, the full classical-method consensus table, the ML diagnosis with confidence, the plain-language SHAP explanation, a risk score with a severity color band (critical/high/medium/low), a recommended action, and an abstention banner where applicable. **70 HTML files** are produced in `reports/per_transformer/`. This single feature is described as moving the project "from 'ML notebook' to 'condition-monitoring tool' in a recruiter's eyes."

Step 8 — Data quality / plausibility validation layer (deployment-ready). A real deployment receives gas readings from a field device, and field devices produce corrupt readings: a sensor stuck at zero, impossible negative values, transcription errors. A system that diagnoses garbage input without warning is dangerous. A validator checks: negative values (impossible), values exceeding IEC 60599 physical plausibility bounds, all-gases-exactly-zero (likely missing data or sensor failure), the dataset's own 0.0001 zero-placeholder convention, and an anomaly pattern (elevated C_2H_2 with very low CH_4/C_2H_4, an unusual pattern worth flagging). The validator returns a clear `PASS` / `FAIL` status with itemized issues rather than crashing or silently diagnosing bad data.

Step 9 — Reproducibility scaffolding + GitHub release (open-source quality). Grounded in the finding that repositories scoring higher on the NeurIPS "ML Code Completeness Checklist" correlate with markedly higher star counts — a repository without pinned dependencies, fixed random seeds, and a one-command run script cannot be trusted or extended by anyone else. Built: a version-pinned `requirements.txt`, a one-command `run.py` that reproduces every key result end to end, GitHub Topics added to the repo (`dissolved-gas-analysis`, `transformer-health-monitoring`, `predictive-maintenance`, `xgboost`, `shap`, `iec-60599`, `condition-monitoring`, `ieee-c57104`, `fault-classification`), a tagged **v1.0.0** release with descriptive release notes, a GitHub Actions CI workflow that verifies the environment on every push, and a basic `pytest` suite covering the classical methods, the validator, and the consensus engine.

Step 10 — Package the Python DGA library (ecosystem contribution). The original strategic research explicitly found: *"No actively maintained Python DGA library exists. Existing DGA tools are an Excel add-in (engineers-tools/xDGA), HTML-canvas Duval tools, or one-off research/MATLAB repos."* To fill this real ecosystem gap, the classical methods (not the ML model) were packaged as a standalone, pip-installable Python library, `pydga`, in its own GitHub repository (`AR0714/pydga`), with a clean public API (`import dga; dga.run_consensus(H2=2240, CH4=1400, C2H6=620, C2H4=280, C2H2=5)`), a proper `pyproject.toml`, an MIT license, a pinned dependency list, a basic test suite, and publication to **PyPI** via `twine upload dist/*` — after which `pip install pydga` works for anyone in the world.

Part 2 Summary Table (from the original handbook)

Step	What Was Built	Why It Matters
1	README rewrite + full repo push	Makes completed work visible to recruiters
2	6-method consensus engine	Mirrors Camlin DGA Matrix — commercial differentiator
3	Abstention / engineer-review layer	Keeps human in the loop — Reinhausen design principle
4	Leakage audit + honest evaluation	Publishable honesty — most projects skip this
5	4-model comparison + macro-F1 + CIs	Rigorous evaluation — accuracy alone was noise
6	SHAP → plain-language evidence	Reinhausen: evidence + probability + recommendation
7	70 automated per-transformer reports	Product behaviour — mirrors TESSA automated reports
8	Gas plausibility validator	Deployment-ready — refuses garbage input
9	Reproducibility + CI + release	Open-source quality standard; NeurIPS checklist
10	`pydga` Python library on PyPI	Fills real ecosystem gap — original contribution

Standing rules that did not change across Part 2 (carried forward from the handbook verbatim): stratified splits always — rare classes must keep their proportion; report per-class metrics (precision/recall/F1), not just overall accuracy; use macro-F1 plus bootstrap confidence intervals on the 70-row test; every model is judged on the sealed, unseen test set only; calibrated probabilities (never raw confidence) feed all risk rankings; never invent data — no time-series, no RUL, no streaming, no dollar-risk values; evaluation honesty is treated as a feature, not a weakness, and every limitation is disclosed explicitly.

6 Part 3 — LLM-Powered Diagnostic Agent (RAG over IEC 60599)

Subtitle: "Project Handbook & Implementation Guide"

Part 3 elevates the project from a standalone ML classifier into a **conversational industrial diagnostic agent**. An engineer can enter dissolved-gas readings, receive a fault diagnosis with its SHAP explanation, and then ask natural-language follow-up questions — with every answer grounded in the actual text of IEC 60599:2022 and IEEE C57.104-2019, not hallucinated facts. The core idea, stated directly in the handbook: **Diagnosis (ML) + Why (SHAP) + What should I do? (RAG over IEC 60599 + LLM) = complete engineering decision support.**

6.1 Technology Stack

Component	Technology	Why Chosen
LLM runtime	Ollama (local)	Free, fully offline, no API key required
LLM model	Llama 3.2 (3B) / Mistral (7B)	Runs on an 8 GB-RAM laptop
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`)	Fast, lightweight, runs on CPU
Vector store	FAISS (CPU)	No server needed, pure Python
RAG orchestration	Custom Python (no LangChain)	Keeps dependencies minimal
Interface	CLI (no Streamlit/FastAPI at this stage)	Matches project scope; kept clean

No LangChain, no OpenAI API, and no internet calls at inference time — the only network call is to Ollama's own `localhost:11434`, on the same machine.

6.2 Architecture Flow

```
Gas inputs → [Input Validation] → [XGBoost + Calibration] → Diagnosis + SHAP
                                                                    ↓
Engineer follow-up question → [Sentence Embeddings] → [FAISS Vector Search]
                                                                    ↓
Top-3 IEC sections retrieved → [Prompt Builder] → [Ollama LLM] → Grounded answer
```

6.3 Ollama Setup (One-Time)

Install Ollama from ollama.com (runs as a background service); verify with `ollama --version`. Pull the model:

```
ollama pull llama3.2       # ~2 GB, fits an 8 GB RAM laptop
# or, on a 16 GB machine, for better answers:
ollama pull mistral
ollama list                # confirms the model is present
```

Ollama must be running whenever the agent is used; it starts automatically after install, or manually via `ollama serve`.

6.4 Additional Dependencies

```
pip install sentence-transformers faiss-cpu --break-system-packages
python -c "import sentence_transformers; print('OK')"
python -c "import faiss; print('OK')"
```

6.5 IEC 60599 Knowledge Base

The knowledge base is built from **plain-text summaries** of the key IEC 60599:2022 and IEEE C57.104-2019 sections, written into text files rather than using the copyrighted PDF directly:

Section	Content	Relevant Fault
IEC 60599 §5.1	Normal operation, monitoring intervals	Normal
IEC 60599 §5.3.1	Partial discharge — H_2 dominant, void/moisture causes	PD
IEC 60599 §5.3.2	Low-energy discharge, sparking, C_2H_2+H_2 markers	D1
IEC 60599 §5.3.3	High-energy arcing, flashover risk, urgent action	D2
IEC 60599 §5.3.4	Thermal < 300 °C, overheated contacts, CH_4 marker	T1
IEC 60599 §5.3.5	Thermal 300–700 °C, winding hotspot, CH_4+C_2H_4	T2
IEC 60599 §5.3.6	Severe thermal > 700 °C, insulation breakdown risk	T3
IEEE C57.104 §4	Action levels L1–L4, concentration thresholds	All
IEC 60599 §6	Ratio methods — Rogers, Key Gas, Duval Triangle	All

6.6 Files Added (Steps 15–19)

Step	File	Purpose
15	`src/build_kb.py`	Chunks IEC text into ~200-token passages, builds a FAISS index + JSON metadata (chunk text, section number, source)
16	`src/rag_engine.py`	Embeds a query and retrieves the top-k (default 3) most relevant IEC passages
17	`src/agent.py`	Calls the Ollama HTTP API (`localhost:11434`) with a prompt combining the system role, diagnosis + SHAP context, retrieved IEC passages, and the user's question; streams the response; handles connection errors gracefully
18	`src/diagnose_agent.py`	Full interactive CLI loop: enter gas values → Part 2 classifier runs → diagnosis + SHAP shown → open-ended Q&A follows
19	`notebooks/15_rag_pipeline.ipynb`	Evaluation notebook: tests the agent on 5 real fault scenarios

6.7 Expected Agent Interaction (verbatim from the handbook)

```
$ python src/diagnose_agent.py

  TRANSFORMER DGA DIAGNOSTIC AGENT
  Powered by XGBoost + SHAP + RAG (IEC 60599)
  Type 'exit' to end session.
========================================

Enter gas readings (ppm):
  H2  : 1200
  CH4 : 85
  C2H6: 40
  C2H4: 950
  C2H2: 210

  Diagnosis  : D2  (High-energy discharge)
  Confidence : 91.3%  [HIGH]

  Top SHAP drivers:
  1. Acetylene (C2H2) — arcing marker  [SHAP +0.74]
  2. C2H2/C2H4 ratio  [SHAP +0.51]
  3. Hydrogen (H2)    [SHAP +0.38]

Ask a follow-up question (or 'exit'):
> What action does IEC 60599 recommend for D2?

  [Retrieving from IEC 60599 knowledge base...]

  Based on IEC 60599 Section 5.3.3, a D2 diagnosis indicates
  high-energy electrical discharge (arcing). The standard
  recommends: immediate offline inspection, investigation of
  flashover risk, and analysis of paper/insulation samples.
  Key marker gases are C2H2, H2, and C2H4.
  [Source: IEC 60599:2022 §5.3.3]
```

6.8 Evaluation Plan (Step 19)

The agent is tested on five scenarios covering every major fault class, checked for whether it cites the correct IEC section and rated for answer relevance on a 1–5 scale:

Scenario	Gas Profile (H_2/CH_4/C_2H_6/C_2H_4/C_2H_2)	Expected Fault	Expected IEC Section
1 — PD	800 / 10 / 5 / 2 / 0	PD	§5.3.1
2 — D1	300 / 50 / 20 / 100 / 80	D1	§5.3.2
3 — D2	1200 / 85 / 40 / 950 / 210	D2	§5.3.3
4 — T1	30 / 200 / 80 / 20 / 0	T1	§5.3.4
5 — T3	150 / 400 / 60 / 800 / 50	T3	§5.3.6

6.9 Git Commit Strategy and Milestone

```
# Step 15: feat: build IEC 60599 knowledge base + FAISS index
# Step 16: feat: add RAG retrieval engine
# Step 17: feat: add Ollama LLM integration
# Step 18: feat: add interactive diagnostic agent CLI
# Step 19: docs: add RAG evaluation notebook + results
git tag -a v2.0.0 -m "feat: LLM-powered RAG diagnostic agent"
git push origin v2.0.0
```

The **v2.0.0** tag marks a full project milestone: calibrated ML classifier (Part 1) + SHAP + IEC guidance + classical methods (Part 2) + LLM-powered conversational agent (Part 3).

---

7 Part 4 — Conversational Digital Twin + Live Web Dashboard

Subtitle: "Conversational Transformer Digital Twin + Live Dashboard"

7.1 The Vision

Part 3 built an LLM diagnostic agent that takes five gas values, runs them through the XGBoost model, retrieves IEC 60599 guidance, and generates an engineering report — but a real user, especially a layperson, should not need to know what H_2 or C_2H_2 means. They should be able to simply ask: *"Is the transformer okay?"* and get a clear, intelligent answer. Part 4 builds two things on top of Part 3:

1. **A Conversational Transformer Agent** — a chatbot where anyone, layperson or engineer, can ask anything about the transformer in natural language and receive a meaningful, context-aware answer grounded in real sensor data.
2. **A Live Web Dashboard** — a browser-based interface showing an animated transformer graphic, real-time health gauges, gas-level charts, and an embedded chatbot panel, deployable online so recruiters and stakeholders can access it from anywhere.

Example conversations envisioned directly in the handbook:

User Asks (Natural Language)	System Responds (Plain English)
Is the transformer okay?	Yes, your transformer is operating normally. Gas levels are within safe limits. Schedule the next check in 6–12 months.
Should I be worried?	There are early signs of partial discharge — a type of electrical fault inside the insulation. It is not dangerous yet, but an inspection within 3 months is recommended.
What is acetylene?	Acetylene is a gas that forms inside a transformer when there is electrical arcing — like a spark inside the oil. High levels mean urgent maintenance is needed.
How long will this transformer last?	Based on current gas levels and thermal stress, the transformer appears healthy. No immediate risk to service life is indicated.
What voltage is it handling?	Currently receiving 11,000 V on the primary side and stepping it down to 433 V for the local distribution network at 68% load capacity.
What happens if we ignore this?	If the partial discharge fault is left unchecked, it can progress to full electrical breakdown (arcing), which may cause a transformer fire or outage. Early action is always cheaper.

7.2 System Architecture — Data Flow

```
USER → types question in chatbot (browser or CLI)
INTENT DETECTOR → understands what the user is asking (health / voltage / gas / advice)
SENSOR READER → pulls latest transformer readings from simulator (or real hardware)
ML MODEL → XGBoost runs on current DGA gas values → diagnosis + confidence
RAG ENGINE → retrieves relevant IEC 60599 / IEEE chunks from FAISS index
LLM (llama3.2) → formats everything into plain English, scaled to user level
RESPONSE → shown in chatbot panel / dashboard / CLI
```

7.3 Phase A — Conversational Backend (Steps 20–24)

Built and tested entirely in the CLI before any dashboard work begins — the correct order is to get the brain working first, then build the face.

Step 20 — Sensor Simulator (`src/sensor_simulator.py`). Since no physical hardware is connected, this module simulates a live data feed, generating readings consistent with the current fault scenario being tested, designed so a real SCADA/Modbus reader could replace it later without changing any other code. Outputs: primary voltage (11 kV / 33 kV / 132 kV), secondary voltage (stepped-down by ratio), load percentage (0–100%, realistic daily variation curve), oil temperature (°C, correlated with load), winding temperature (°C, typically 15–25 °C above oil under load), the five DGA gas readings (H_2, CH_4, C_2H_6, C_2H_4, C_2H_2 — able to simulate any fault class), and a cumulative operating-hours counter.

Step 21 — Conversation Memory (`src/transformer_memory.py`). Maintains a short-term memory of the conversation within a session. Without memory every message is treated independently and the chatbot cannot answer follow-up questions; with memory, a user can say "tell me more about that" and the system understands the referent. Memory stores: the last 8 conversation turns (user message + assistant response pairs), the last diagnosis result and confidence level, the last sensor snapshot (so follow-up questions use consistent data), and the session start time plus turn count.

```
User:  Is the transformer okay?
Agent: Early signs of partial discharge detected. Confidence 75%.
User:  What does partial discharge mean?
Agent: [knows from memory the last diagnosis was PD — explains in context]
User:  How urgent is it?
Agent: [still referencing PD — gives timeline and risk assessment]
```

Step 22 — Intent Detector (`src/intent_detector.py`). Classifies what the user is actually asking so the chatbot gives the right type of answer and pulls the right data — a voltage question needs electrical readings; a fault question needs the ML model output.

Intent Class	Example User Phrases	Data Needed
`health_status`	"Is it okay? Is transformer healthy? Any problems?"	Model diagnosis + confidence
`voltage_query`	"What voltage? Primary/secondary? Step up or down?"	Sensor voltage readings
`gas_query`	"What is acetylene? Why is H2 high? Explain C2H2"	RAG IEC knowledge base
`temperature`	"Is it overheating? What is the oil temperature?"	Temperature sensor readings
`load_query`	"How much load? Is it overloaded? Capacity?"	Load percentage reading
`action_advice`	"What should I do? Should I shut it down? Urgent?"	Diagnosis + IEC action levels
`life_estimate`	"How long will it last? Remaining life?"	Cumulative stress indicators
`explain_fault`	"What is partial discharge? What does D2 mean?"	RAG IEC knowledge base
`general_chat`	"Hello / Thanks / What can you tell me?"	None — conversational response

Step 23 — Core Chatbot Brain (`src/transformer_chatbot.py`). The central orchestrator of Part 4, wiring together the sensor simulator, intent detector, XGBoost model, RAG engine, conversation memory, and Ollama LLM. For every user message it follows a fixed eight-step pipeline:

1. Detect intent using `intent_detector`
2. Pull the latest sensor snapshot from `sensor_simulator`
3. If the intent needs diagnosis, run XGBoost on the current DGA gases
4. Retrieve relevant IEC 60599 chunks from FAISS via `rag_engine`
5. Build a prompt combining sensor data, diagnosis, IEC context, and conversation history
6. Send the prompt to Ollama `llama3.2` and get a plain-English response
7. Store the turn in conversation memory
8. Return the response to the user

A **layperson vs. engineer mode** is also detected automatically from vocabulary — a question using the word "acetylene" gets a technical answer; "is it okay?" gets a simple yes/no with a one-sentence explanation.

Step 24 — CLI Chatbot + Evaluation (`src/chat_cli.py`). An interactive command-line chatbot for testing the full pipeline before the dashboard is built, with a formal evaluation of 10 test questions covering every intent class, verified for quality and correctness:

```
$ python src/chat_cli.py
Transformer Agent ready. Ask me anything.
You: Is the transformer okay?
Agent: Your transformer is showing early signs of partial discharge...
```

The 10 evaluation questions: *"Is the transformer working normally? Should I be worried about anything? What voltage is the transformer handling? Is it overheating? What is acetylene and why does it matter? How loaded is the transformer right now? What should the maintenance team do? Is there any urgent risk? How long until the next inspection? Tell me more about partial discharge."* Success criteria: all 10 questions answered without errors; layperson questions get simple, jargon-free responses; technical questions get detailed engineering responses; follow-up questions correctly use conversation memory. After evaluation passes, all Phase A files are committed and tagged **v3.0.0-alpha**.

7.4 Phase B — Live Web Dashboard (Steps 25–27)

Wraps the Phase A backend in a Flask web server and delivers a browser dashboard with an animated transformer, live gauges, and the embedded chatbot panel.

Step 25 — Flask Server (`app/server.py`). A lightweight Flask server serving the dashboard HTML and exposing three API endpoints:

API Endpoint	Method	Returns
`/`	GET	Serves `index.html` dashboard page
`/api/sensors`	GET	Latest sensor snapshot as JSON — voltage, load, temp, gas ppm
`/api/chat`	POST	Accepts a user message, returns the LLM response as JSON
`/api/diagnosis`	GET	Latest ML diagnosis — class, confidence, reliability, probabilities

```
python app/server.py
# * Running on http://localhost:5000
```

Step 26 — Dashboard (`app/static/index.html` + CSS + JS). The dashboard is divided into four visual zones on a single page: - **Zone 1 — Top banner:** project title, transformer name/ID, a green online-status dot. - **Zone 2 — Left panel:** an animated transformer SVG graphic with current-flow lines, step-up/step-down labels, and live primary/secondary voltage readouts. - **Zone 3 — Center panel:** a health status badge (NORMAL / WARNING / CRITICAL in green/yellow/red), a confidence meter, the fault class in plain English, six gas-level bars (H_2, CH_4, C_2H_6, C_2H_4, C_2H_2), and load/temperature gauges. - **Zone 4 — Right panel:** the chatbot interface — scrollable message history, a text input box, a Send button, and a "powered by" label.

The animated transformer graphic uses two coil symbols (primary and secondary windings), animated dashed current-flow lines on the primary side, a stepped-down-voltage arrow on the secondary side, a core color that shifts green/yellow/red with health status, and animation speed that increases with load percentage. JavaScript polls `/api/sensors` every 3 seconds and `/api/diagnosis` every 5 seconds, updating all gauges smoothly without a page refresh; the chatbot calls `/api/chat` on message submission.

Step 27 — Deployment.

```
Add requirements.txt with flask, sentence-transformers, faiss-cpu, joblib, requests
Add a Procfile: web: python app/server.py
Push to GitHub
Connect GitHub repo to Render — auto-deploys on every push
Add the live URL to GitHub README and LinkedIn profile
```

Important note preserved from the handbook: Ollama runs locally only — for cloud deployment, the LLM call switches to a free hosted API (Groq / Together AI / Hugging Face Inference) with the same `llama3.2`-class model, requiring zero change to the chatbot logic, only a changed HTTP endpoint. This is precisely the design decision that was later carried through into the live **GridPulse** deployment, which calls **Groq's `llama-3.3-70b-versatile`** model through a Netlify serverless function rather than a local Ollama instance.

Platform	Cost	Best For	URL Example
Local (`localhost:5000`)	Free	Demo on your own laptop	`http://localhost:5000`
Render.com	Free tier	Online — share with recruiters	`transformer-dga.onrender.com`
Railway.app	Free tier	Fastest deploy — push to GitHub	`transformer.up.railway.app`
Hugging Face Spaces	Free	ML project showcase	`huggingface.co/spaces/AR0714/...`

7.5 Technology Stack — Part 4

Component	Technology	Purpose
Sensor data	Python (custom simulator)	Simulates real-time transformer readings
ML diagnosis	XGBoost + joblib	Fault classification from DGA gases (Part 2)
Knowledge retrieval	FAISS + sentence-transformers	RAG over IEC 60599 (Part 3)
LLM (local)	Ollama `llama3.2`	Generates plain-English responses
LLM (cloud)	Groq API (`llama3-8b-8192` -class)	Free cloud LLM for the deployed version
Web server	Flask (Python)	Serves dashboard + chat API
Frontend	HTML + CSS + JavaScript	Animated dashboard, no framework needed
Charts/gauges	Chart.js (CDN)	Gas-level bars, load gauge, history chart
Deployment	Render.com	Free cloud hosting, auto-deploy from GitHub

7.6 Four Important Notes (carried verbatim)

1. Build Phase A completely before touching Phase B — the chatbot must work perfectly in the CLI before adding the dashboard.
2. The sensor simulator uses the exact same feature vector as the XGBoost model (H_2, CH_4, C_2H_6, C_2H_4, C_2H_2) — the order must never change.
3. Ollama must be running (`ollama serve`) whenever testing locally; for cloud deployment, switch to the Groq API.
4. The Flask server and Ollama can run simultaneously on the same machine, since they use different ports (5000 and 11434). All new Part 4 files go under `src/` (backend) or `app/` (web) — existing Part 2/3 files are never modified.

7.7 Part-by-Part Project Status (as recorded in the Part 4 handbook)

Part	Title	Status	Tag
Part 1	Data Loading, EDA, Classical DGA Methods	COMPLETE	—
Part 2	XGBoost + Calibration + SHAP + CLI Predictor	COMPLETE	v1.0.0
Part 3	LLM Diagnostic Agent with RAG over IEC 60599	COMPLETE	v2.0.0
Part 4	Conversational Digital Twin + Live Dashboard	IN PROGRESS	v3.0.0

8 GridPulse — The Production Front-End

The Part 4 vision — a live, animated, browser-based transformer dashboard with an embedded conversational assistant — was ultimately realized as **GridPulse**, a standalone, cinematic React + Three.js application, deployed live and publicly accessible.

- **Live site:** transformer-dga.netlify.app
- **Repository:** github.com/AR0714/gridpulse-dashboard

What it shows: a ~25,000-particle 3D copper winding coil carrying a glowing current pulse; a central **Health Index orb** (0–100) whose flame-ring edge changes character with the active fault — calm amber for Normal, blue-white corona sparks for partial discharge, orange tendrils for low-thermal faults, a white-hot rim for severe thermal faults, red lightning arcs for high-energy arcing (D2); a live DGA feed of all seven gases re-read every 3 seconds with realistic jitter, checked against IEC 60599 typical limits; a **dissolved-gases modal** showing TDCG, per-gas share, the key gas, an interpretation, the top SHAP-style drivers, and a side-by-side comparison against the Duval triangle; three IEC 60599/IEEE C57.104-grounded maintenance-action recommendations per fault; and a **diagnostic assistant** chat panel answering questions about the unit, powered by Llama 3.3 70B via Groq through a secure Netlify serverless function (falling back to rule-based answers when no API key is configured — the API key is never shipped to the browser).

A **UNIT-07 Substation profile** feature mirrors the familiar "click your profile photo to see a bigger view" UX pattern from everyday apps: clicking the small circular substation avatar in the top bar opens a centred glassmorphism modal with a larger hero photograph and the unit's technical specifications — voltage level (132/33 kV), transformer capacity (2 × 40 MVA), year commissioned (2009), ambient temperature, physical location (Bhubaneswar, Odisha), and last inspection date — alongside the live Health Index bar, closing on Escape or a backdrop click, with full keyboard/focus-trap accessibility.

Five fault scenarios are switchable live with keys **1–5** or on-screen pills: **N** (Normal), **PD** (partial discharge), **T1** (thermal < 300 °C), **T3** (thermal > 700 °C), **D2** (high-energy arcing). The dashboard is full-bleed at every screen size (fills exactly 100vw × 100vh with no scrollbars), respects `prefers-reduced-motion`, lazy-loads the 3D scene after the UI paints, and caps pixel ratio adaptively for performance. The model results quoted on the GridPulse README are pulled directly from the research repository: **80.0%** calibrated XGBoost accuracy vs. **57.1%** for the classical Duval triangle (+23 points), with an honest note that the underlying sensor data on the live dashboard is simulated for demonstration, while the model metrics themselves come from the real research project documented above.

---

9 Consolidated Results

Method	Accuracy (70-row sealed test)	Notes
Key Gas (classical)	~60%	Single dominant gas only
IEC 60599 Ratios (classical)	Partial	No-decision on ~30% of cases
Duval Triangle (classical)	**57.1%**	Industry standard since the 1970s
Random Forest (ML baseline)	78.6%	With physics-informed features
XGBoost — calibrated (ours)	**80.0%**	ECE: 0.15 → 0.10 after isotonic calibration

+23 percentage points above the Duval Triangle baseline. Both methods evaluated on the same 70-row held-out test set (10 samples per class, stratified split). Additional findings: SHAP confirms the model relies most heavily on acetylene (C_2H_2) to detect arcing (SHAP value +0.74 on the demonstrative D2 case); isotonic calibration reduced the expected calibration error (ECE) from 0.15 to 0.10; a full leakage audit found 41 of 49 IEC TC10 benchmark rows (84%) duplicated in training, corrected to an honest 8-row unseen evaluation; bootstrap 95% confidence intervals across SVM, Random Forest, XGBoost, and LightGBM confirmed that the 1–2 point accuracy differences between tree-based models are statistically indistinguishable noise at this sample size.

10 Honest Limitations

Carried forward verbatim from the project's own README and evaluation-honesty sections, because disclosing limitations explicitly is treated throughout this project as a mark of engineering maturity, not a weakness to hide:

- The dataset is approximately 400 labelled samples drawn from overlapping public sources. Reported figures are dataset-specific and should not be read as real-world field performance.
- The rarest class (T2, thermal 300–700 °C) has only 21 samples in the full dataset; per-class recall for this class is correspondingly unstable.
- There is no time-series data, so trend-based features — which the standards themselves weight heavily for diagnosing degradation rate — are entirely absent from this system.
- The system has not been validated against any physically inspected transformer outside the benchmark set.
- The sealed 70-row test set is curated (10 samples per class), not a natural real-world class distribution, and so it likely overstates performance relative to an imbalanced real deployment.
- The IEC TC10 benchmark, as originally obtained, had 41 of 49 rows (84%) overlapping with the training data; only 8 genuinely unseen rows remain after correction, which is too small a sample for strong independent claims.
- This project is explicitly a diagnosis-and-triage layer built on top of classical standards — not a real-time sensor platform, not a remaining-useful-life predictor, not a digital twin in the full sense, and not a source of financial-risk figures. These were deliberately excluded rather than fabricated.
- On the live GridPulse dashboard specifically, the sensor readings shown are simulated for demonstration purposes; the model accuracy and evaluation figures quoted are the real, validated results from the underlying research repository.

11 Final Repository Structure

```
transformer-health-dga/
├── data/
│   ├── raw/                          # Original DGA dataset
│   └── iec_knowledge_base/           # 9 IEC 60599 text files (Part 3)
├── models/
│   ├── xgb_dga_base.joblib           # Raw XGBoost model (Part 2)
│   ├── xgb_dga_calibrated.joblib     # Isotonic-calibrated model (Part 2)
│   └── faiss_index/                  # FAISS vector index (Part 3)
│       ├── index.faiss
│       └── metadata.json
├── src/
│   ├── classical/                    # key_gas.py, iec_ratios.py, duval.py,
│   │                                 # rogers.py, doernenburg.py, duval_pentagon.py,
│   │                                 # consensus.py (Parts 1 & 2)
│   ├── validation/
│   │   └── gas_validator.py          # Plausibility validation layer (Part 2)
│   ├── reporting/
│   │   └── report_template.py        # Per-transformer HTML reports (Part 2)
│   ├── calibration.py                # CalibratedDGAModel class (Part 2)
│   ├── predict.py                    # CLI predictor (Part 2)
│   ├── build_kb.py                   # Builds FAISS index (Part 3)
│   ├── rag_engine.py                 # RAG retrieval engine (Part 3)
│   ├── agent.py                      # Ollama HTTP integration (Part 3)
│   ├── diagnose_agent.py             # CLI diagnostic agent (Part 3)
│   ├── sensor_simulator.py           # Live sensor data (Part 4)
│   ├── transformer_memory.py         # Conversation memory (Part 4)
│   ├── intent_detector.py            # Intent classification (Part 4)
│   ├── transformer_chatbot.py        # Conversational brain (Part 4)
│   └── chat_cli.py                   # CLI chatbot (Part 4)
├── app/                              # Web dashboard (Part 4)
│   ├── server.py                     # Flask web server
│   └── static/
│       ├── index.html
│       ├── transformer.css
│       └── dashboard.js
├── notebooks/
│   ├── 01_load_and_look.ipynb        # Parts 1 & 2 worked here
│   └── 15_rag_pipeline.ipynb         # Part 3 RAG evaluation
├── reports/
│   ├── calibration_curve.png
│   ├── shap_summary_bar.png
│   ├── shap_waterfall.png
│   ├── confusion_matrix.png
│   ├── risk_coverage_curve.png
│   ├── per_class_recall_heatmap.png
│   ├── fleet_risk_ranking.csv
│   └── per_transformer/              # 70 automated HTML reports
├── tests/                            # pytest suite
├── requirements.txt                  # Pinned dependency versions
├── run.py                            # One-command reproduction script
├── Procfile                          # Cloud deployment
├── DECISIONS.md                      # Running engineering decision log
└── README.md

gridpulse-dashboard/                  # Production front-end (separate repo)
├── src/
```

```
│   ├── scene/                        # Three.js winding coil, pulse, dust
│   ├── ui/                           # TopBar, HealthOrb, GasModal,
│   │                                 # SubstationProfileModal, Assistant, etc.
│   ├── lib/                          # dga.ts, looks.ts, assistantContext.ts
│   ├── store/useMonitor.ts
│   └── data/scenarios.ts
├── netlify/functions/chat.ts         # Groq-backed AI assistant (server-side key)
└── docs/demo.gif                     # Live interactive demo

pydga/                                # Standalone open-source library (separate repo)
├── dga/
│   ├── classical/                    # Pure-Python re-implementation
│   └── validation/
└── pyproject.toml                    # Published to PyPI
```

12 How to Run Everything

Research pipeline (`transformer-health-dga`):

```
pip install -r requirements.txt
jupyter notebook notebooks/01_load_and_look.ipynb
# or, for one-command full reproduction:
python run.py
```

RAG diagnostic agent (Part 3):

```
ollama pull llama3.2
ollama serve                # if not already running
python src/diagnose_agent.py
```

Conversational dashboard (Part 4):

```
python app/server.py
# open http://localhost:5000
```

Production dashboard (`gridpulse-dashboard`):

```
npm install
npm run dev                 # http://localhost:5173
# optional: add a free Groq key at console.groq.com/keys to .env for live AI chat
```

Standalone classical-methods library:

```
pip install pydga
python -c "import dga; print(dga.run_consensus(H2=2240, CH4=1400, C2H6=620, C2H4=280,
C2H2=5))"
```

13 Standards and References

- **IEC 60599:2022** — Mineral oil-impregnated electrical equipment in service: guide to the interpretation of dissolved and free gases analysis.
- **IEEE C57.104-2019** — IEEE Guide for the Interpretation of Gases Generated in Mineral Oil-Immersed Transformers (supersedes the 2008 edition).
- **CIGRE Technical Brochure 761** — DGA interpretation guidance referenced for classical-method validation.
- **Dataset:** IEEE DataPort, DOI: 10.21227/27vy-h479, plus the IEC TC10 benchmark set used for leakage-aware evaluation.
- **Kapoor, S. & Narayanan, A. (2023).** "Leakage and the reproducibility crisis in machine-learning-based science." *Patterns*, 4(9):100804 (Princeton) — basis for the Part 2 data-leakage audit methodology.
- **Sutikno, T. et al. (2024).** *Heliyon*, 10(4):e25975 — basis for the multi-method classical consensus engine design.
- **Commercial systems studied for design inspiration:** GE Vernova Kelman, Hitachi Energy TXpert, Camlin Technologies TOTUS/DGA Matrix, Reinhausen TESSA.

14 Credits

Prepared by: Ankit Raj, B.Tech Electrical Engineering (Final Year), KIIT University, Bhubaneswar. **Visual design** of the GridPulse front end adapted from an "artery" health-app concept on Dribbble, reworked for a power transformer. **This documentation** merges and preserves, in full, the content of four internal project handbooks: *Handbook for the Transformer DGA Project* (Part 1, 25 steps), *Intelligent Transformer Health Monitoring — Part 2 Handbook: From Working Model to Industrial-Grade Product* (10 steps), *Part 3 — LLM-Powered Diagnostic Agent Handbook* (RAG over IEC 60599), and *Part 4 Handbook — Conversational Transformer Digital Twin + Live Dashboard* (Phases A & B).

This documentation is intended for publication across the project's GitHub repositories, LinkedIn, and personal portfolio site.